

On-the-fly Range-Velocity Radar Data Compression Exploring Doppler Spectrum Redundancy

Zoran Zivkovic, Liang Li

NXP Semiconductors
High Tech Campus 46, Eindhoven, The Netherlands
email: zoran.zivkovic@nxp.com, liang.li@nxp.com

Abstract: Radar systems extracting target range and velocity use sequence of pulses in Pulse-Doppler systems, or sequence of short chirps for Frequency Modulation Continuous Wave (FMCW) radar. In both cases large amount of intermediate data needs to be stored before extracting the velocity information from the Doppler spectrum. An on-the-fly compression scheme is presented here that can reduce the needed storage space. The technique explores the redundancies in the Doppler spectrum. Basic compression architecture is described to take advantage of the nature of the data-generation process. A number of implementation options are proposed and evaluated on simulated and on real data. The proposed method is lossless. Practical “lossy” versions with fixed compression rate are also derived which allow graceful degradation of the performance and are suitable for embedded implementation.

1. Introduction

The Pulse-Doppler radar has been used for many years to simultaneously estimate the distance and radial velocity of the detected objects [1]. Multiple pulses are transmitted and the received echoes binned over the range. The velocity can be extracted from the multiple pulses in each range bin. Velocity information allows discrimination between objects moving at different speeds, e.g. an aircraft, a vehicle or the background clutter. The Frequency Modulation Continuous Wave (FMCW) radar with short chirps is becoming popular for simultaneously measuring the distance and the radial velocity [2,3,4]. In both cases the nature of the data acquisition and processing is similar. If the system is using M range measurements and N range bins, before extracting the velocity information the whole matrix containing $M*N$ values needs to be stored in the memory, as illustrated in Figure 1. Phased array radars use multiple antennas to extract angular information. If the system has L antennas the total number of bins that need to be stored becomes $L*M*N$. It is also anticipated that the storage needs will increase for modern systems:

- The radar data has typically large dynamic range, so with increasing requirements in the future, more bits will be needed per bin
- Better resolution will increase the number of elements for all three dimensions L , M and N of the stored data

For example, in a system with $M=1024$ velocity bins and $N=1024$ range bins and $L=32$ antennas, the memory space becomes 256MB, where we assumed 32+32 bit representation per complex number per bin. The large memory can increase the costs of a radar system especially for fully integrated solutions. Furthermore, the memory readout speed limitations can introduce latency in detecting targets.

The radar data is typically sparse [5] and range-velocity data contains a lot of redundancy and potentially can be compressed. Furthermore, in most radar system only the target positions and velocities are of interest, therefore the mentioned memory space is just an intermediate buffer needed to store the intermediate processing results.

On-the-fly compression of the memory content is common for many high performance systems [6]. A radar specific scheme is proposed here. While there are many options to explore the data redundancy, exploring the Doppler spectrum is the most natural fit to the typical range-velocity radar systems. The classical compression techniques such as Bzip or LZO could be also applied but they do not exploit the nature of the radar data. Image based techniques, such as JPEG, could be used on the full data matrix, but this would not be on-the-fly compression. The technique proposed here is more effective as it will be shown. This paper is organized as follows. First, the general compression scheme is described. Practical implementation choices are discussed next. Afterwards, some simulation and real data experiments and evaluation are presented. The final section contains the conclusions and the further work topics.

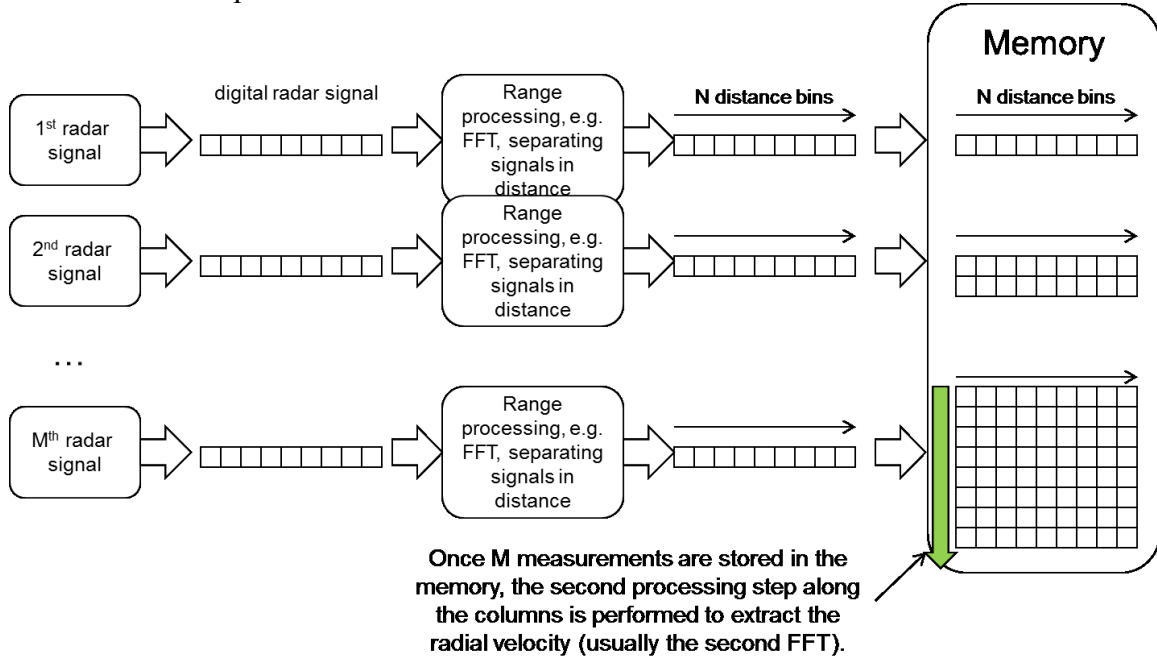


Figure 1. Data acquisition and processing basic structure for a typical range-velocity radar system.

2. Basic lossless compression scheme

Typical data acquisition and processing flow is described in Figure 1. In the “short chirp” FMCW case [2] the range processing is the first FFT for each range measurement. The velocity is extracted by the second FFT pass over the columns. An example of simulated FMCW data is shown in Figure 2. The data compression and decompression blocks are added to reduce the size of the memory needed for the intermediate data before the second FFT pass. If the objects are detected also on-the-fly after performing the second FFT, the intermediate data is the only large amount of data that needs to be kept in the memory. Reducing the size of this memory buffer by compression techniques presented here can largely reduce the costs of the radar system and often also increase the speed of processing if the memory data reading is a limiting factor. The basic elements of the compression scheme are illustrated in Figure 3. Details for each block are discussed next.

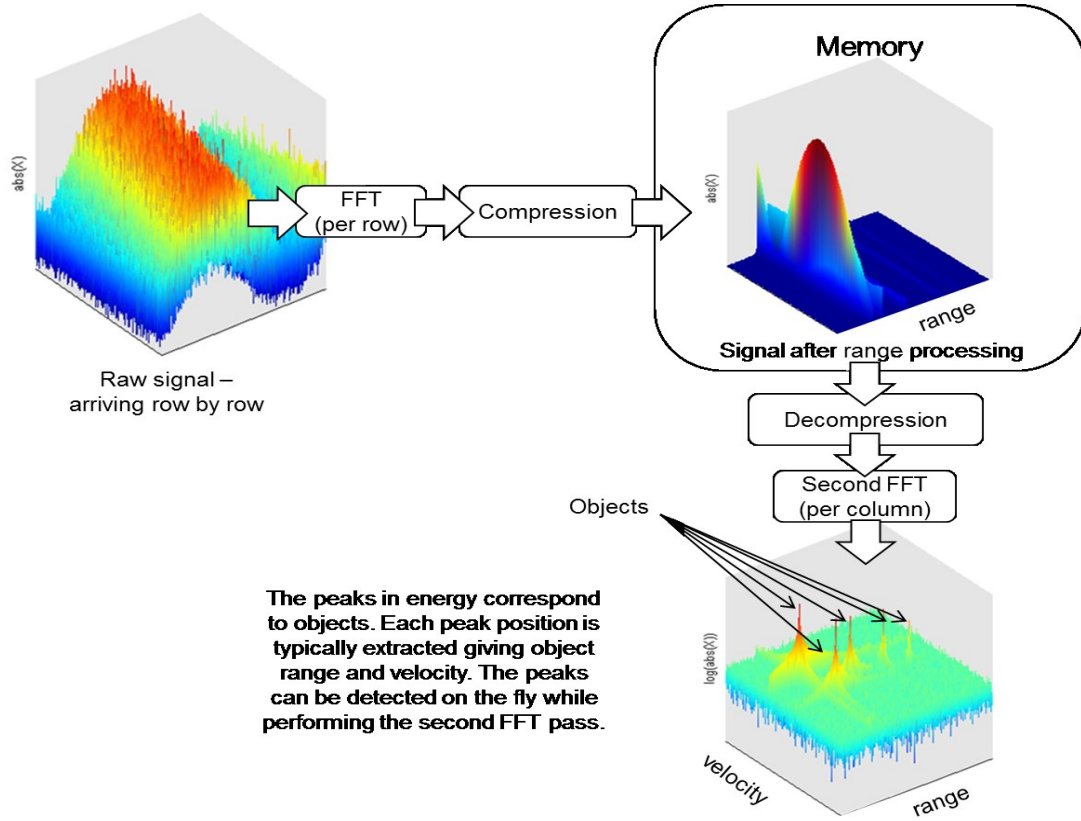


Figure 2. Example FMCW simulated data processing with on-the-fly data compression and decompression.

Let each distance measurement have N bins/elements, and let M be the number of measurements performed for the speed estimation. After the range processing, e.g. the first FFT, a measurement m consists of N complex numbers $X(m, 1), \dots, X(m, N)$. The measurement represents a row of the full $M \times N$ measurement matrix. A column n corresponds to a distance. The compression steps are the following. First, each incoming complex value is compared to the model prediction, as in Figure 3. The prediction is in the amplitude and phase form:

$$\hat{X}[m, n] = \hat{a}[m, n] e^{j\hat{\theta}}$$

where $\hat{a}$ and $\hat{\theta}$ are the predicted magnitude and phase respectively. For the magnitude:

$$\hat{a}[m, n] = \alpha \hat{a}[m-1, n] + (1-\alpha) a[m-1, n]$$

the current predicted value $\hat{a}[m, n]$ is a linear combination of previous predicted value $\hat{a}[m-1, n]$ and the last incoming magnitude value $a[m-1, n]$. For the phase prediction:

$$\begin{aligned} \hat{\theta}[m, n] &= \theta_{mdl}[m, n] + \theta_{incMdl}[m, n] \\ \theta_{mdl}[m, n] &= \theta[m-1, n] \\ \theta_{incMdl}[m, n] &= \beta \theta_{incMdl}[m-1, n] + (1-\beta)(\theta[m-1, n] - \theta_{mdl}[m-1, n]) \end{aligned}$$

The phase prediction $\hat{\theta}$ is the sum of the phase model value θ_{mdl} and the increment model θ_{incMdl} . The phase model is simply the last phase of the incoming data. The phase increment model is a linear combination of the last value of the increment model and the difference between the last incoming data and the last model value. The two predictors act like two IIR filters with tunable parameters, namely α and β . Intuitively, the two parameters control how fast the predictions are influenced and modified by the real input signal.

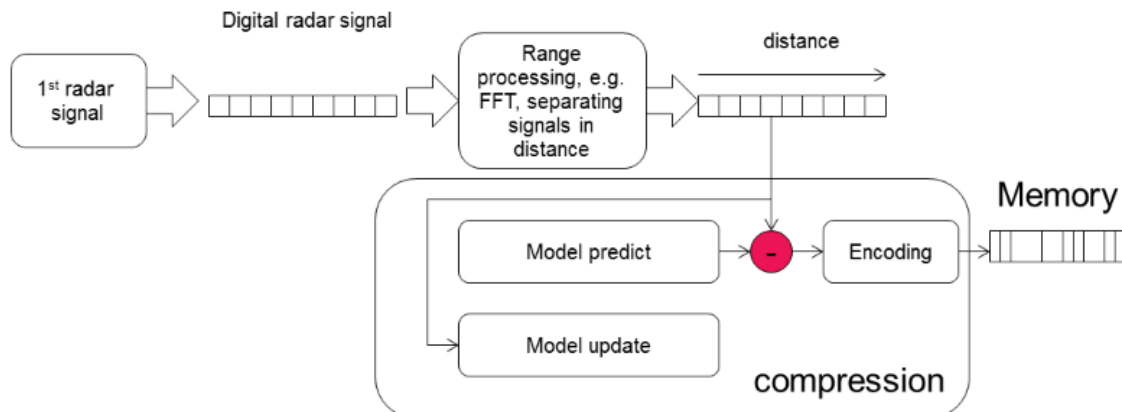


Figure 3. Basic elements of the on-the-fly compression scheme.

The difference between the measured data and the prediction $X[m,n] - \hat{X}[m,n]$ is further encoded and saved to the memory.

3. Encoding

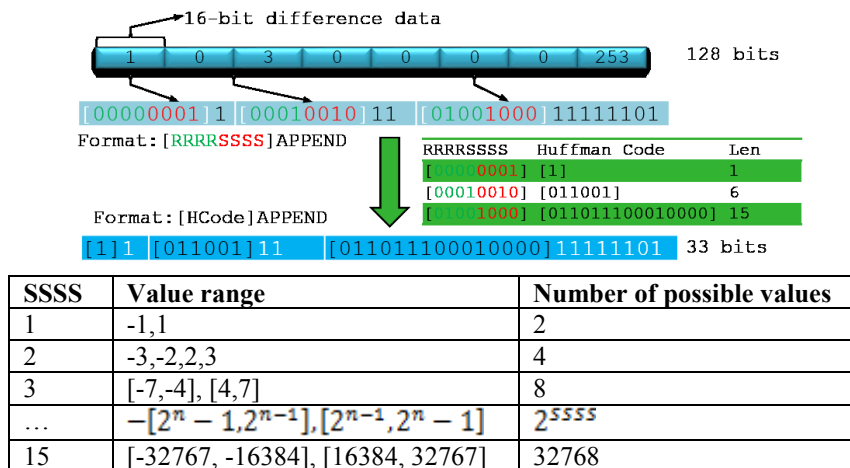


Figure 4 Example of encoding the differences

There are two main characteristics of the difference data. The first is that there are many zeros in the data. The second is that small values are much more frequent than large values. Here we will apply encoding similar to the encoding used in the JPEG standard to explore these data characteristics.

Figure 4 shows the encoding procedure with the example input difference data [1, 0, 3, 0, 0, 0, 0, 253] (each is a 16-bit value). Each value in the input data stream will be encoded into a binary form of [RRRRSSSS+APPEND] where RRRRSSSS is an 8-bit field and APPEND is an SSSS-bit field. For convenience, the name R4S4 might be used instead of RRRRSSSS in the following text. The 4-bit RRRR field in R4S4 is the number of zeroes preceding a non-zero data while SSSS is the region that the non-zero data is in and is also the number of bits that APPEND needs to encode the exact data value.

Table in Figure 4 gives the SSSS values and corresponding ranges. The length of the APPEND field bits is variable but the R4S4 field are fixed (8 bits) where Huffman coding is used.

4. Practical implementation with efficient memory addressing

Due to the use of the Huffman encoding, the length of the encoded data is variable and the total length of each column is not the same as illustrated in Figure 5 (top). Proper memory management has to be implemented if we want to reach high memory usage efficiency and fast readout of the compressed data. The principle is to reserve memory for future use as illustrated in the example presented in Figure 5 (bottom).

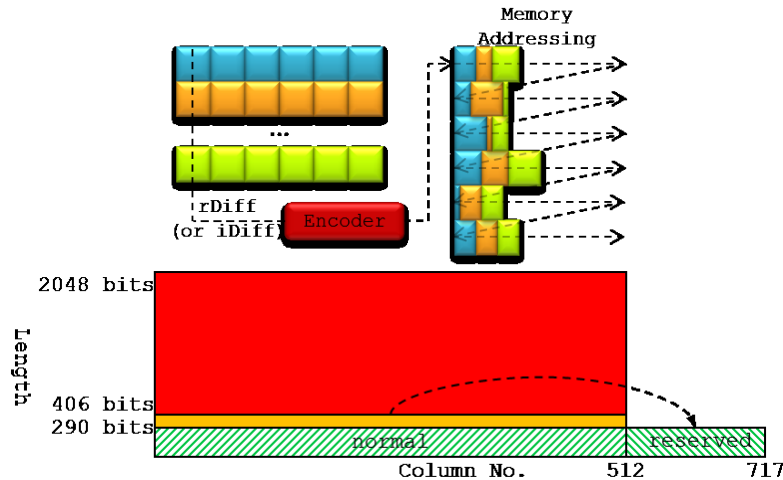


Figure 5. Variable data length per column (top), example memory reshaping for efficient addressing (bottom).

The uncompressed data in this example has 512 columns, each with 2048 bits of data represented by the red rectangle. The smaller rectangles, yellow and green, represent the target compression ratio of 5, in this case it translates to 406 bits per column. Since the column length is variable, some of the columns might exceed the reserved space. Therefore extra space is reserved for overflowed columns, i.e. the yellow region is reserved and is appended to the end of the normal memory region (the area of yellow region and right-inclined filled green region on left side are the same). So the memory actually has 717 columns of 290 bits space resulting in the fixed compression ratio of 5.

The presented memory addressing allows efficient data readout but not fully efficient use of the memory since some of the columns might not be completely filled. Furthermore, the lossless compression cannot be guaranteed if fixed memory size is reserved. If the complete memory is filled, some data points at some of the columns need to be skipped. This will introduce reduction in the signal to noise ratio for detection. The target compression ratio can be tuned to achieve the desired performance, see examples in the next section.

4. Results

Simulated and real data was used to evaluate the compression scheme. Two simulated scenarios were used. First, a simple scenario where 10 targets were randomly placed with random speeds. The complex scenario has multiple object on purpose located at the same distance but with different speeds. This makes the Doppler spectrum more complex and difficult to compress/predict by the simple model used here. For the real data, a 79GHz radar with 1 GHz bandwidth chirp was used. The real radar data was collected on a street with a moving vehicle and sometimes pedestrians present. The achieved compression ratios are given in Table 1. For comparison we applied a number of lossless compression algorithms: LZ0, Gzip, WinZip. For the proposed compression ideal lossless compression ratio is reported first. In all cases the scheme presented here largely outperforms any of the standard compression tools. The reason is that the nature of the data is better explored in this radar signal specific scheme, even though a fixed Huffman encoding table is used. The fixed encoding table is constructed based on simulated data and used in all presented cases.

	Simulated data (simple)	Simulated data (hard)	Real data
Ideal (excluding memory addressing)	9.8	7.3	9.7
Practical lossy (target 5) - (% data loss)	5.0 - (0%)	5.0 - (0.04%)	5.0 - (0%)
Practical lossy (target 6) - (% data loss)	6.0 - (0%)	6.0 - (1.8%)	6.0 - (0.005%)
Practical lossy (target 7) - (% data loss)	7.0 - (0.005%)	7.0 - (3.0%)	7.0 - (0.42%)
LZO (level 1-fast)	2.9	2.7	3.3
LZO (level 9-slow-high compression)	5.7	4.9	6.2
WinZip	6.5	5.5	7.5
Gzip	6.7	5.8	7.7

Table 1. Compression results. The percentage of lost data points is reported for the practical “lossy” implementation with the target fixed compression ratio within brackets. All other schemes are lossless.

The ideal lossless compression scheme could be applied off-line, but it is impractical for real-time processing purposes. The practical “lossy” implementation with efficient memory addressing is applied using a number of fixed target compression ratios. Table 1 lists percentage of the data points that could not fit into the memory for each target ratio. The lost data introduces proportional reduction of the signal to noise ratio in the Doppler spectrum. The compression ratios and the amount of lost data vary depending on the data. The most difficult is the “hard” simulated case that deliberately contains a complex Doppler spectrum.

4. Conclusions

Data compression technique tailored to the range-velocity radar systems is described. The data redundancy in the Doppler spectrum is explored. The data is compressed on-the-fly after the “range” processing and before the “velocity” processing step to reduce the memory size needed to store the intermediate data. This simple technique is shown to be able to provide large compression factors outperforming common general purpose compression schemes. A practical efficient implementation with fixed target compression ratio is presented that gives also competitive compression factors, typically with small data losses.

Basic elements outlined here can be further improved. First, the prediction model can be made more elaborate or tailored for specific situation. In case of phased array radar with multiple antennas the model could explore the redundancy between antennas as well. Second, various other adaptive encoding schemes could be applied. Finally, the memory addressing can be further extended for practical compression ratios closer to the ideal case.

Acknowledgements

We would like to thank Feike Jansen, Artur Burchard and Alessio Filippi for the discussions.

References:

- [1] M.I. Skolnik, "Radar handbook.", 1970.
- [2] A. Stove, “Linear FMCW Radar Techniques,” IEE Proceedings Radar and Signal Processing, UK, 1992.
- [3] H. Rohling and M. Kronauge, “Waveform Design for Automotive Radar”, Chapter 10 in Waveform Design and Diversity for Advanced Radar Systems,” in Institution of Engineering and Technology (IET), 2012.
- [4] M. Kronauge and H. Rohling. "New chirp sequence radar waveform." Aerospace and Electronic Systems, IEEE Transactions on 50.4 (2014): 2870-2877.
- [5] J. Ender. "A brief review of compressive sensing applied to radar." Radar Symposium (IRS), 2013 14th International. Vol. 1. IEEE, 2013.
- [6] R. B. Tremaine, P. A. Franaszek, J. T. Robinson, C. O. Schulz, T. B. Smith, M. E. Wazlowski and P. M. Bland, "IBM memory expansion technology (MXT)", IBM Journal of Research and Development, vol. 45, no. 2, pp. 271-285, 2001.